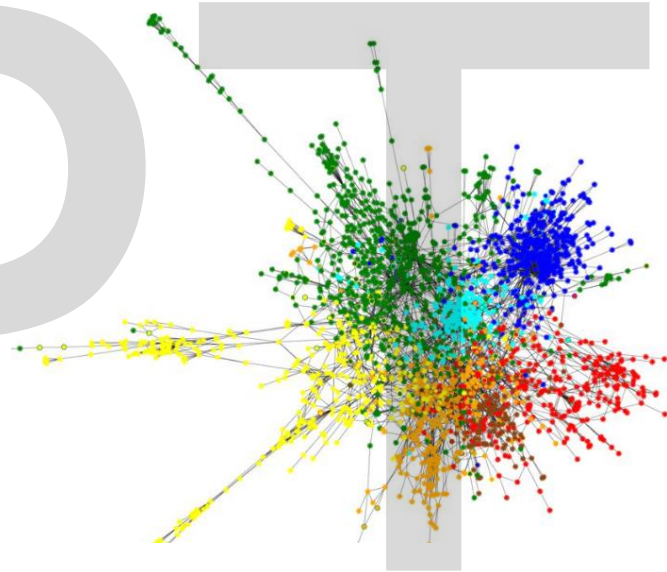


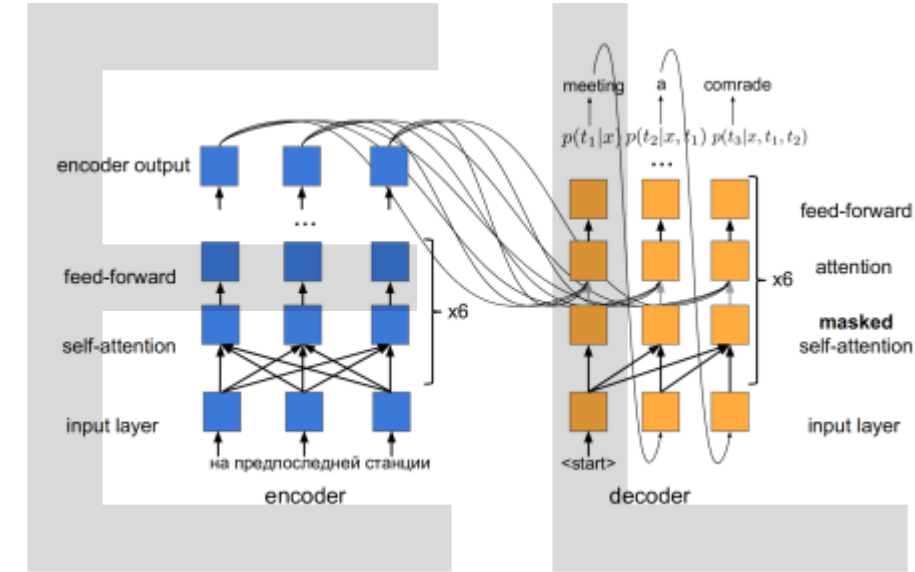
# Story Does Not End Here

- **Complex algorithms**
  - Graphs
  - Transformer algorithms
- **Hardware challenges**
  - Power
  - Fabrication
  - Packaging

Example of Graph

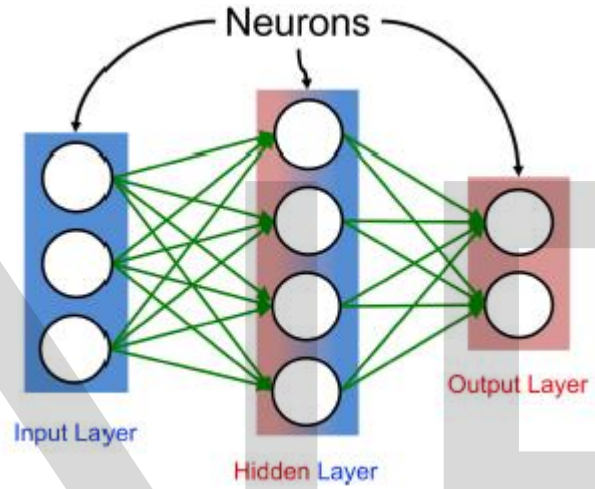


Example of Transformer Algorithm

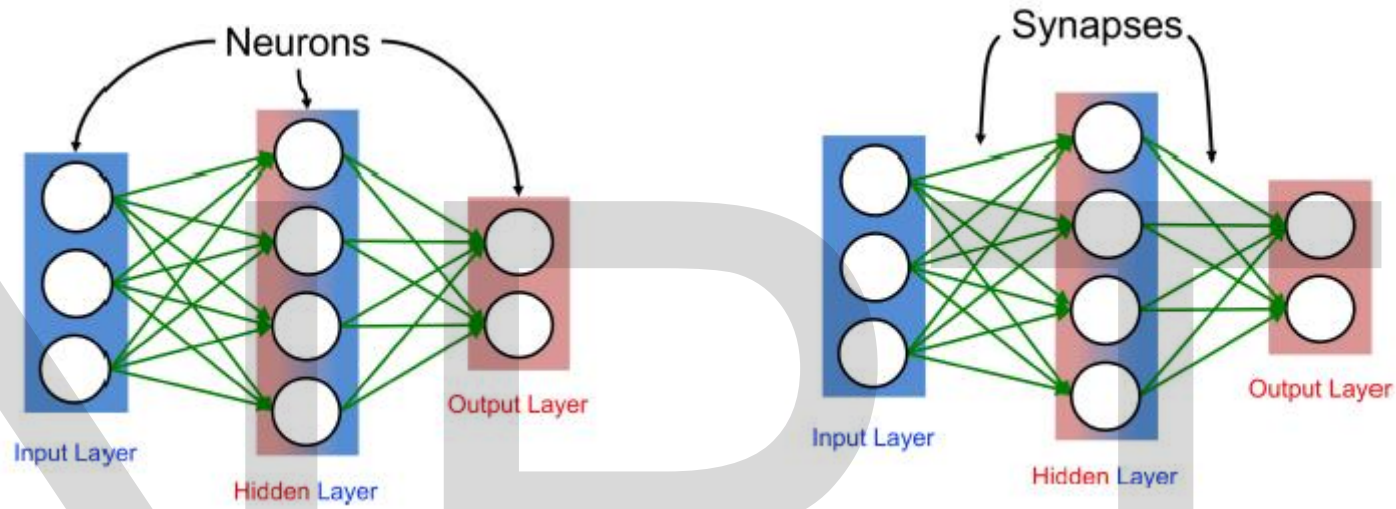


**Will End up Deep Diving on These Topics**

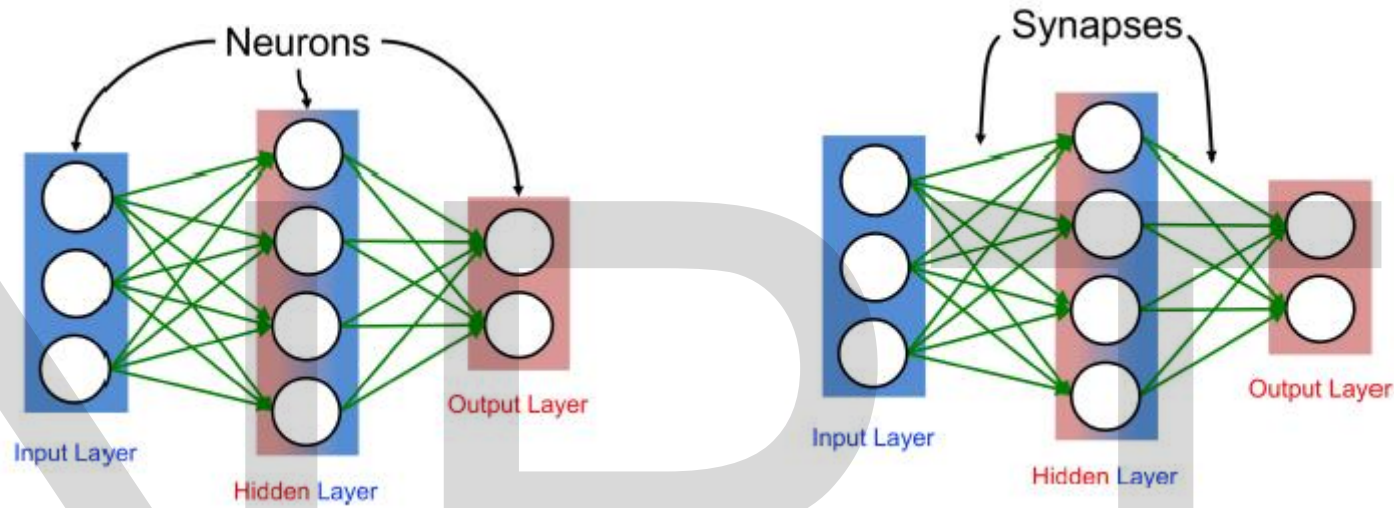
# DNN Terminology



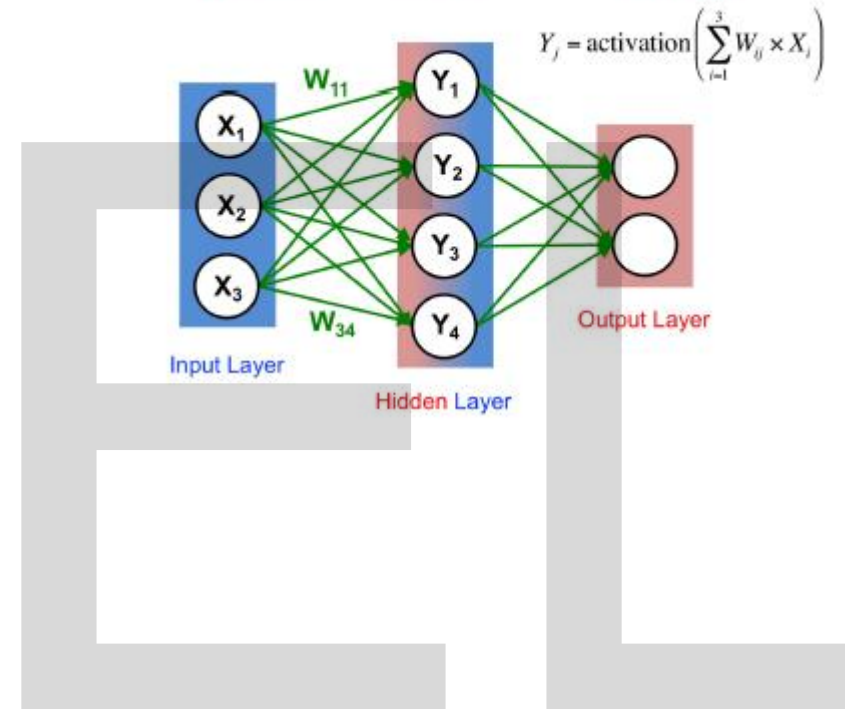
# DNN Terminology



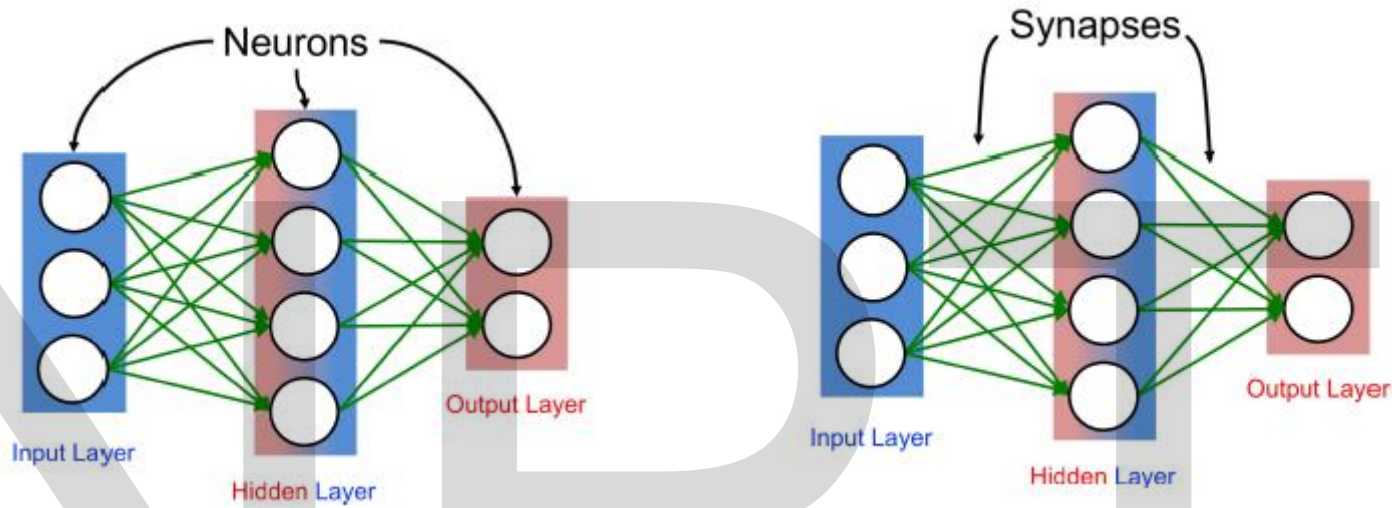
# DNN Terminology



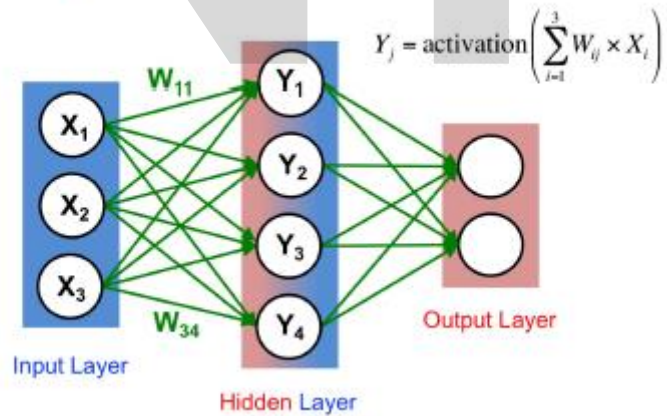
Each synapse has a **weight** for neuron **activation**



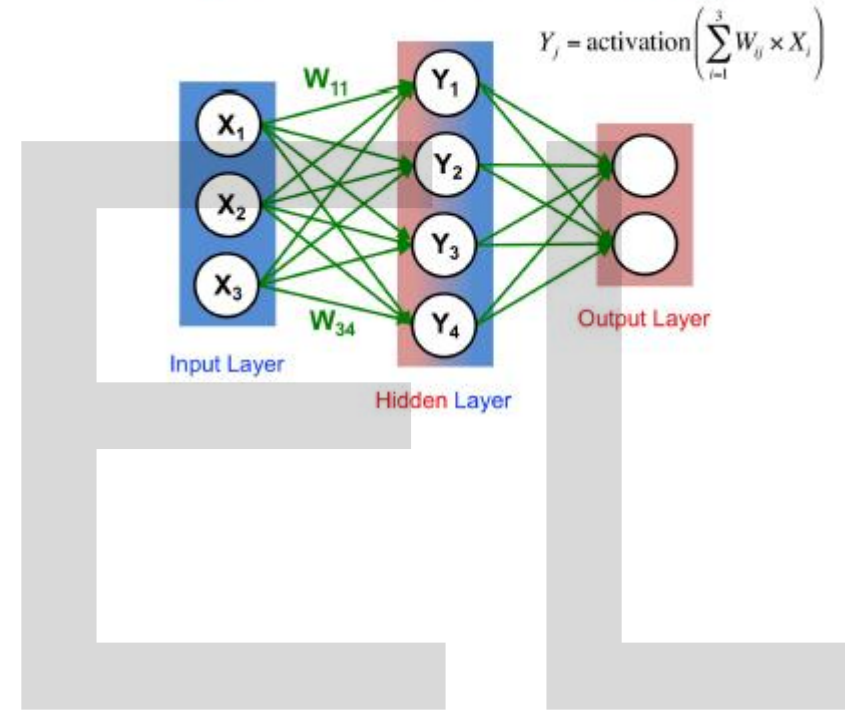
# DNN Terminology



**Weight Sharing:** multiple synapses use the same weight value

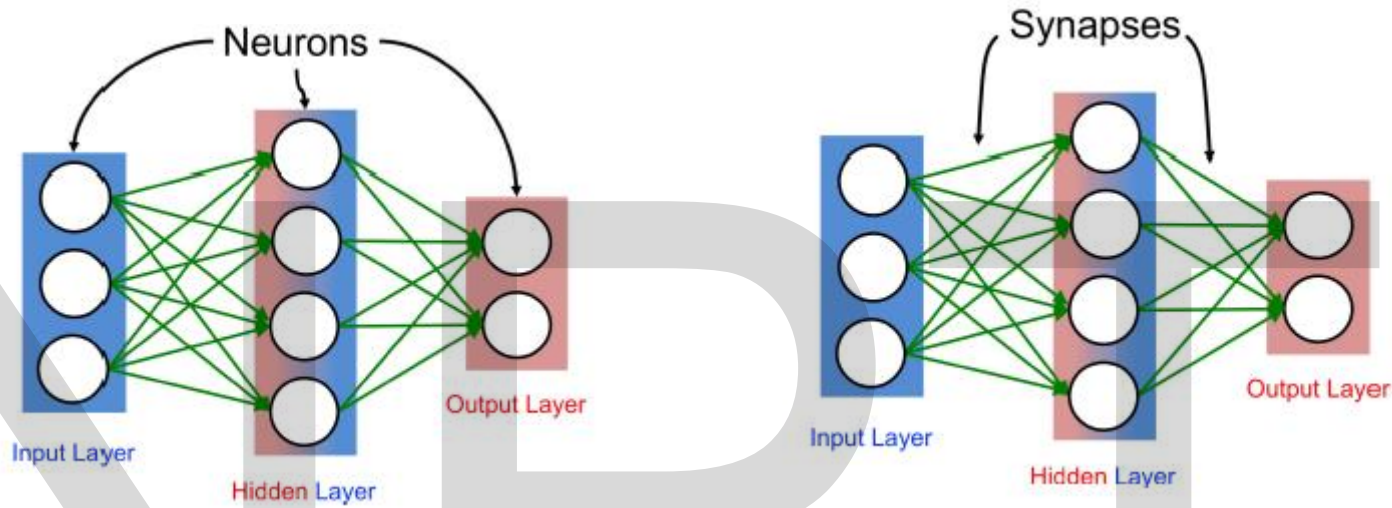


Each synapse has a **weight** for neuron **activation**

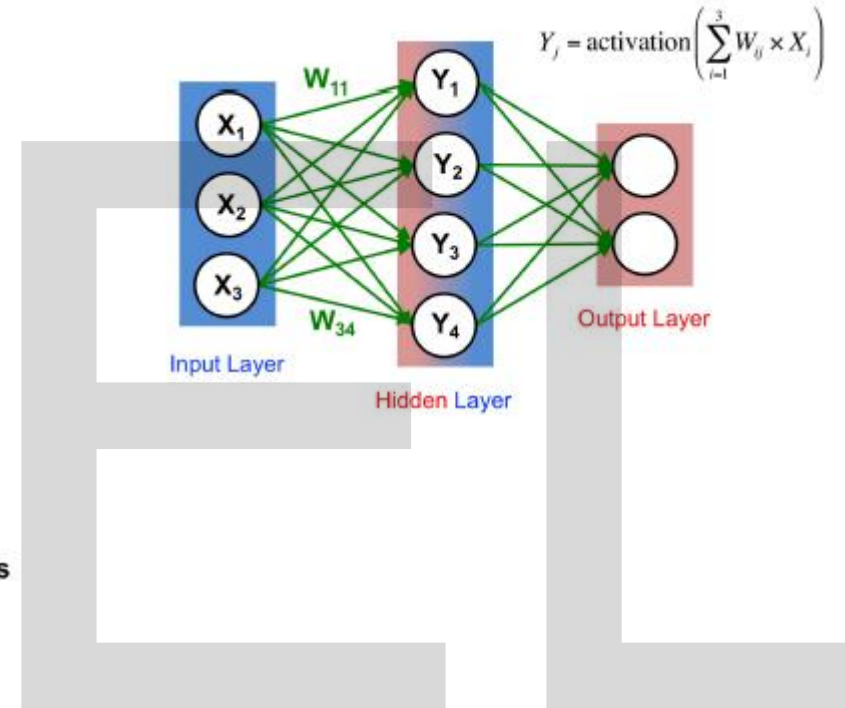




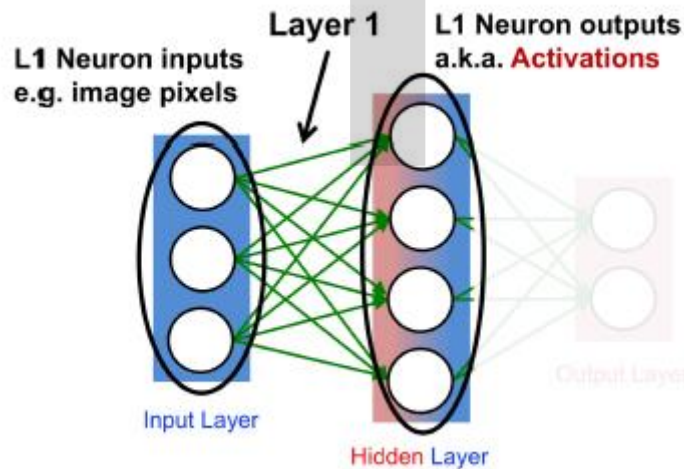
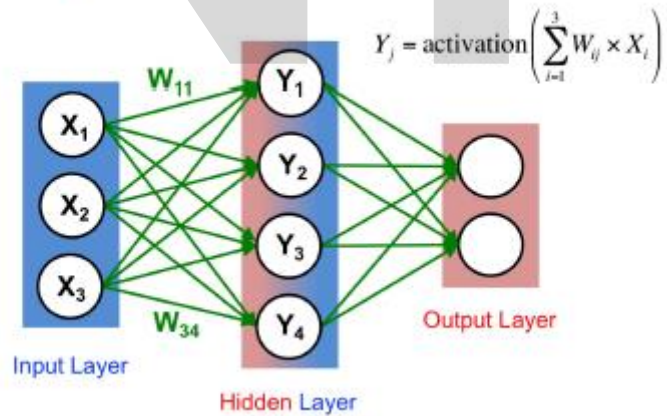
# DNN Terminology



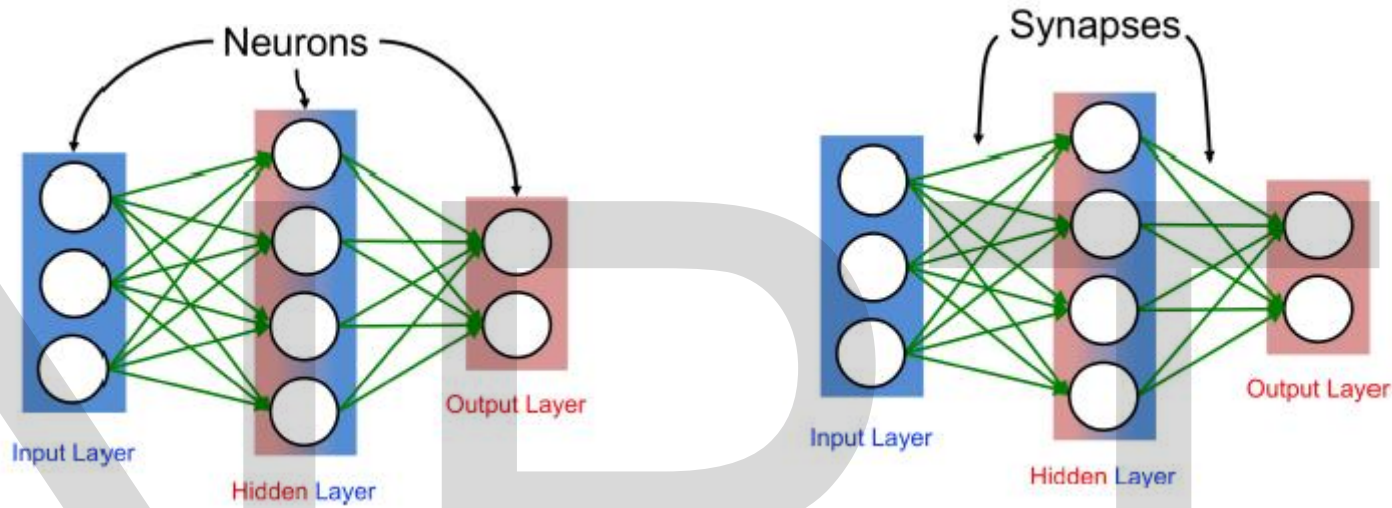
Each synapse has a **weight** for neuron **activation**



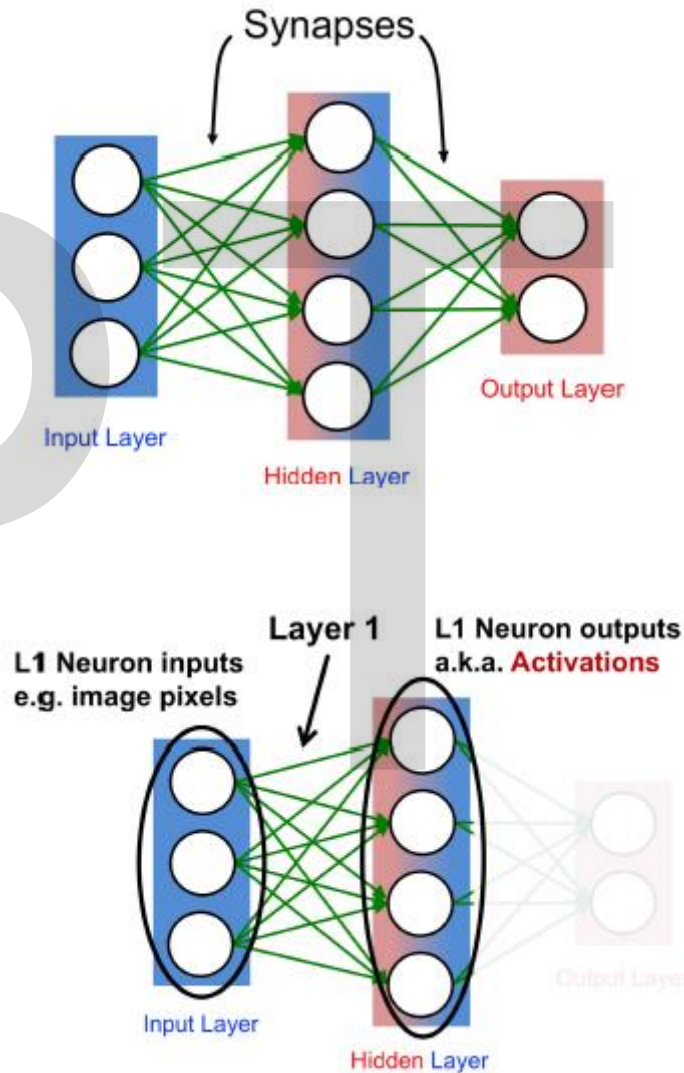
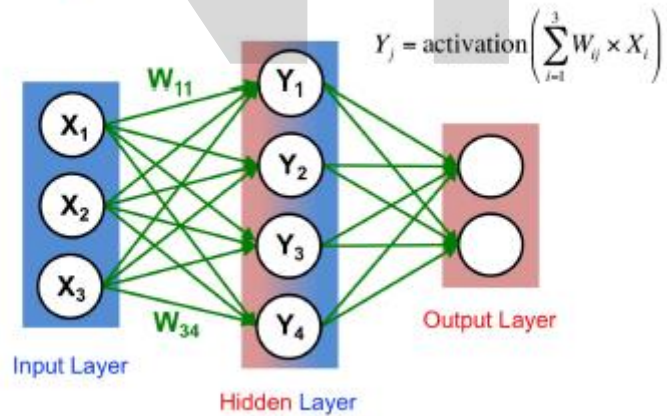
**Weight Sharing:** multiple synapses use the **same weight value**



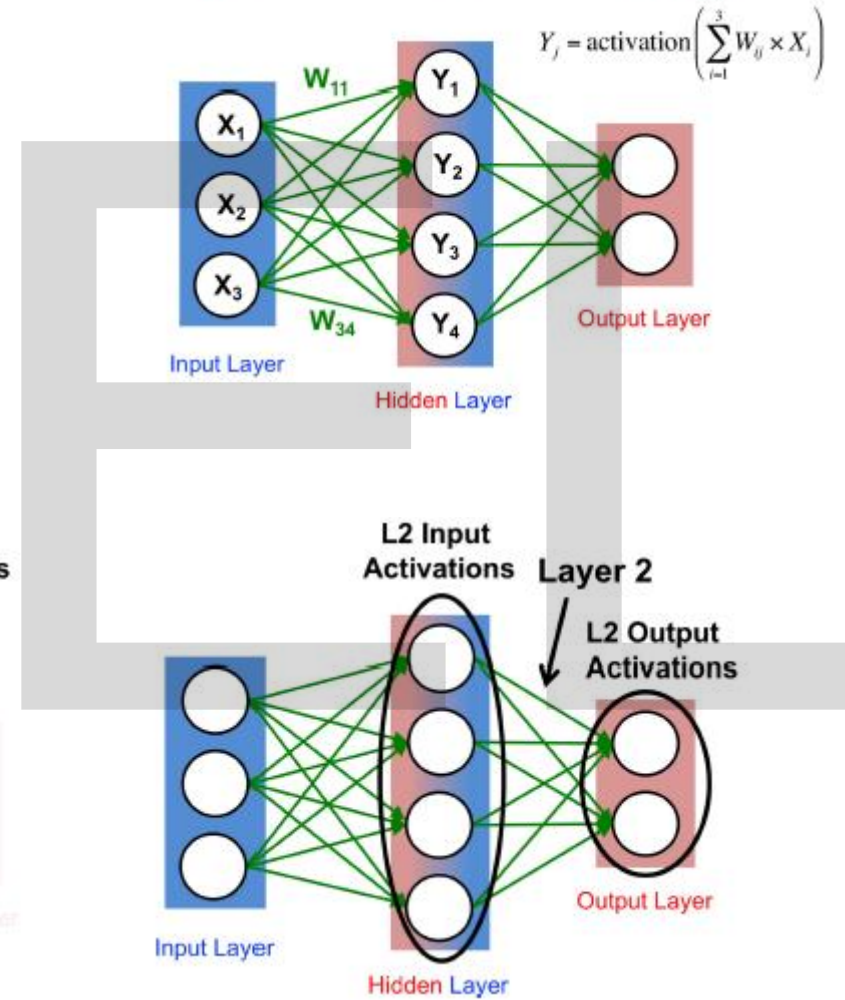
# DNN Terminology



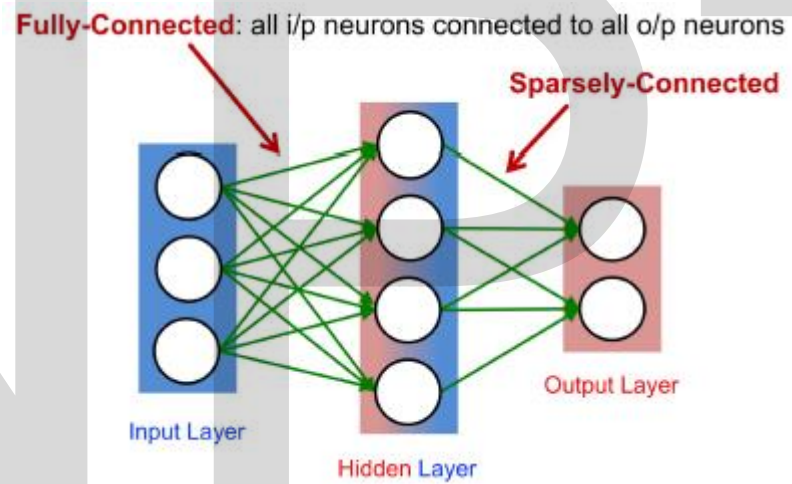
**Weight Sharing:** multiple synapses use the same weight value



Each synapse has a **weight** for neuron **activation**

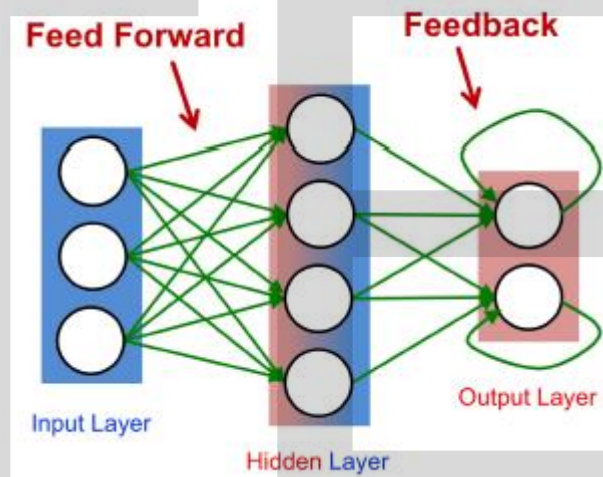
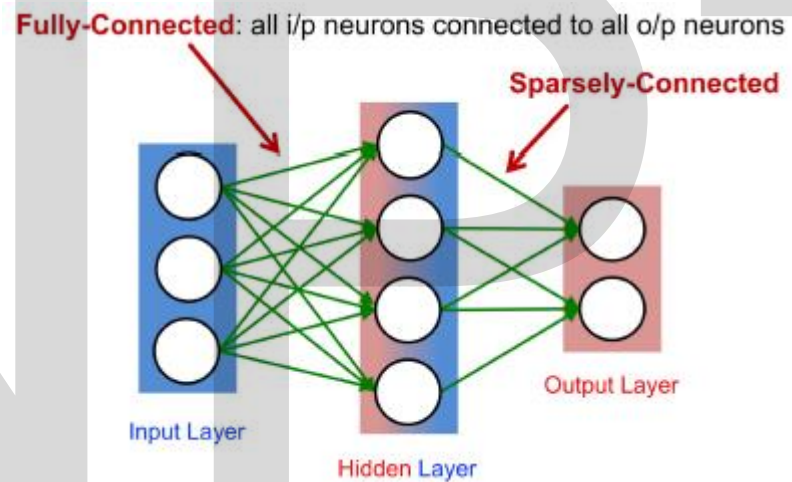


# DNN Terminology





# DNN Terminology



# Typical Types of DNNs

- **Fully-connected NN**
  - Feed forward (also known as multi-layer perceptron)
- **Convolutional NN**
  - Feed-forward, sparsely connected with weight sharing
- **Recurrent NN (RNN)**
  - Feedback
- **Long Short-term Memory (LSTM)**
  - Feedback + storage

# Inference vs Training

- **Inference:** use trained weights to predict
- **Training:** to determine weights
  - Supervised
  - Semi-supervised
  - Unsupervised/self-supervised
  - Reinforcement

# Some Key Observations in DNN Training

- **Partial derivatives share computations**
  - There are common terms for every neuron in each layer
- **Partial derivatives need the intermediate values (activation) obtained during the forward propagation**
  - Re-using the activation



# Vanishing Gradients

- Instead of gradient descent, stochastic gradient descent (SGD) is used to update weights
  - Faster convergence
- SGD works well for just a few layers of NNs
- As backpropagation continues into many layers, the gradient multiplication with weight lead to very small values
  - Since weights are usually of small values
- This problem is known as vanishing gradient problem
- Solutions
  - Residual Connections
  - Rectifier Linear Unit (ReLU) instead of sigmoid (the range of value is less)
  - Batch normalization

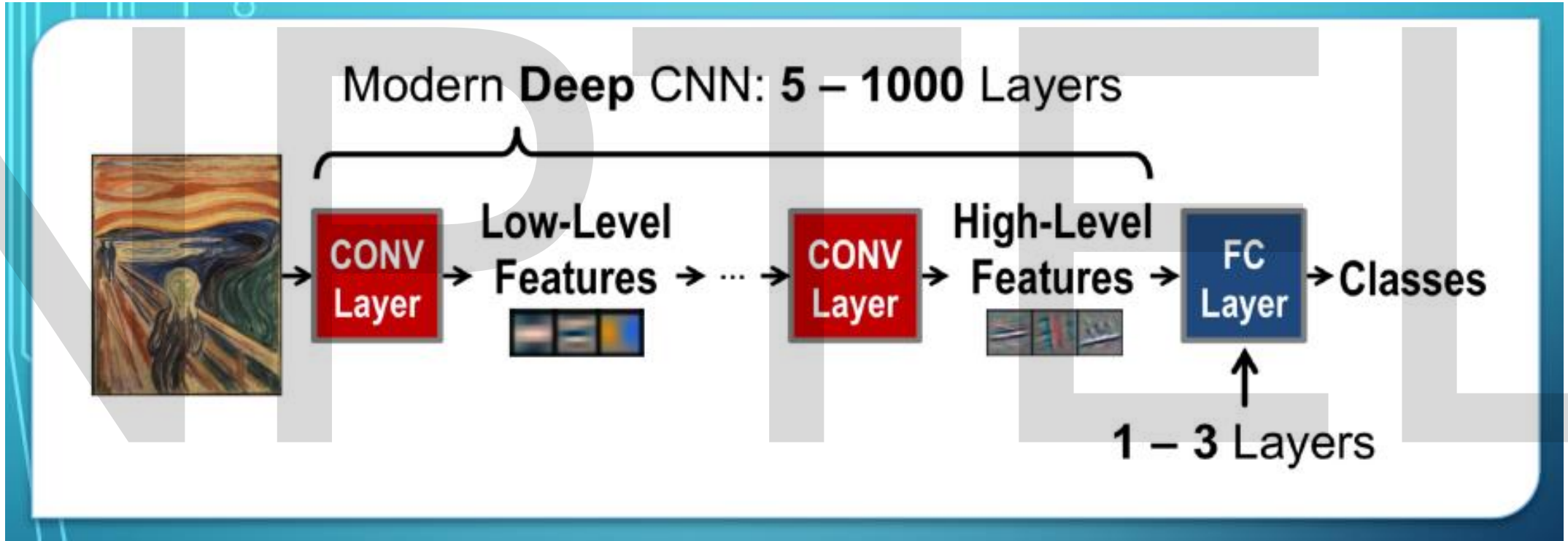
# Relevance to Computer Hardware

- Repeated computations can be avoided
  - Caching
- Identical computation can be reused
- We will eventually see many hardware ML accelerators exploit these properties

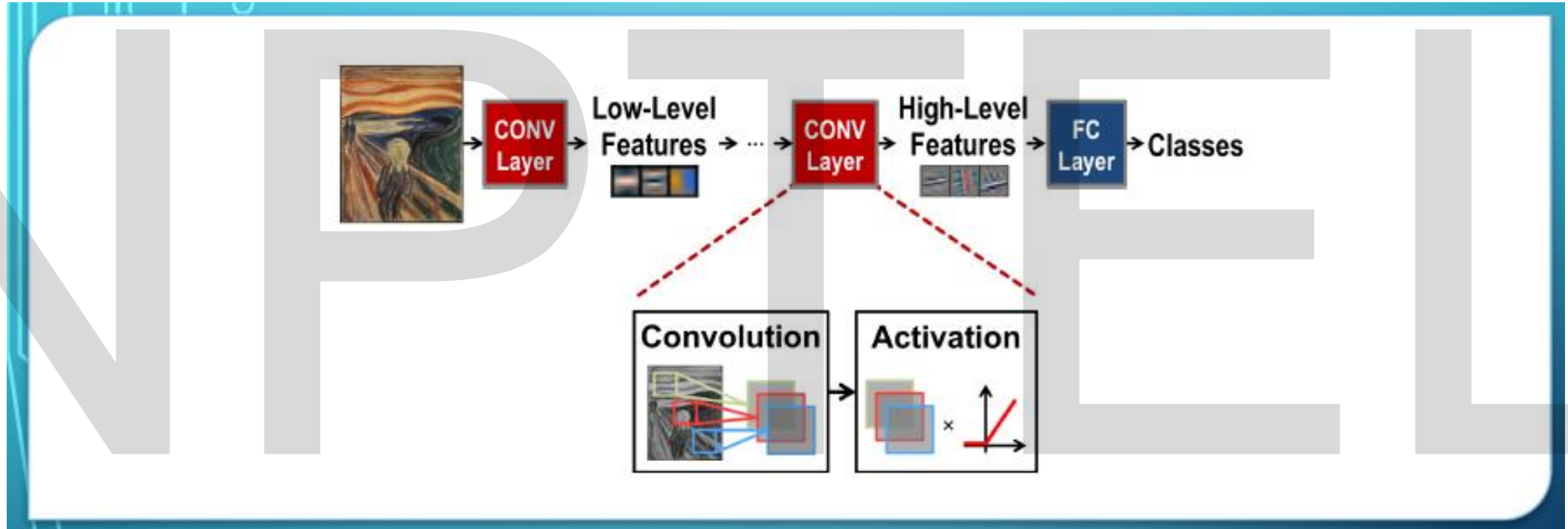
# Convolutional Neural Networks (CNNs)

- **DNN treats each pixel in an image as input to a separate neuron**
  - But it misses the spatial similarity image data
- **Why not capture information from the neighbouring pixels**
  - Convolution!
- **Convolutions**
  - Slide a small matrix of parameters (known as filter) across the input image
  - Train the filter using backpropagation

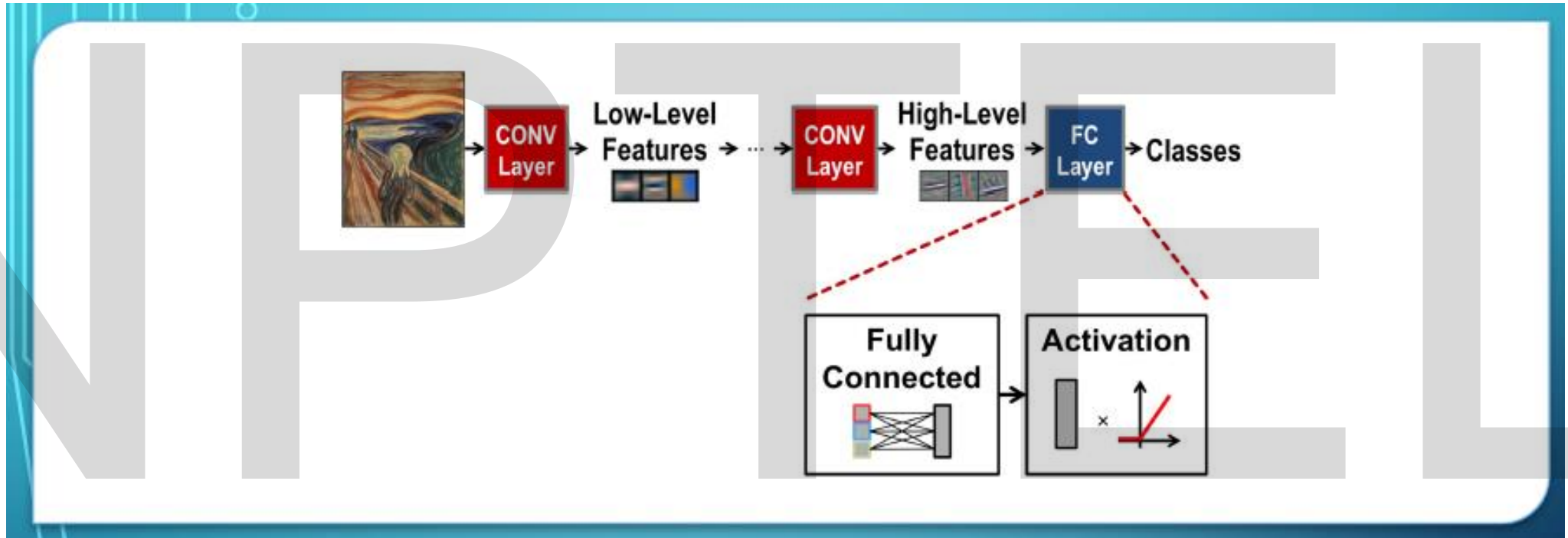
# Deep CNNs



# Deep CNNs



# Deep CNNs





# Deep CNNs

